

Matriz de Optimización de Costos

Qué modelo de IA se usa por tarea, y cuánto ahorra el prompt caching

1. Matriz de decisión: qué modelo por tarea

El workflow no usa un único modelo para todo: cada tarea de IA está asignada al modelo que mejor equilibra costo y calidad para ese trabajo puntual. La tabla siguiente muestra esa decisión y, para cada una, la alternativa que se evaluó y por qué se descartó — no es simplemente "limitar tokens", es una elección de modelo por tarea.

Tarea	Modelo elegido	Por qué este modelo	Alternativa evaluada y por qué no
Clasificación (categoría + urgencia)	Claude Haiku 4.5	Salida corta y de formato fijo (dos campos, sobre listas cerradas de opciones). No requiere razonamiento profundo ni redacción — prioriza velocidad y costo mínimo.	Claude Sonnet 4.5: mismo resultado, pero ~3× más caro por token (sección 2) sin mejora perceptible de calidad en una clasificación de opciones cerradas.
Redacción de la respuesta al cliente	Claude Sonnet 4.5	Tarea de "lectura densa": combina el mensaje del cliente con contexto de FAQ (RAG ligero) y debe producir una respuesta empática y coherente en lenguaje natural. Se beneficia de prompt caching porque el bloque de instrucciones + FAQ se repite entre tickets.	Claude Haiku: más barato, pero calidad de redacción notablemente menor para tono y empatía frente al cliente — no es la cara visible del negocio para arriesgar ahí.
Reprocesamiento masivo / reportes históricos (no implementado aún)	Batch API (Anthropic)	50% de descuento en input y output frente a la API síncrona — ideal para picos de volumen sin restricción de tiempo de respuesta (detalle en sección 6).	No se usa en el flujo actual: cada ticket necesita aprobación humana en minutos (HITL en Slack) y el Batch API entrega resultados en una ventana de hasta 24 hs — incompatible con esa latencia.

2. Precios vigentes por modelo

Precios por millón de tokens, verificados contra los valores reales configurados en las variables de n8n del workflow (PRECIO_HAIKU_INPUT, PRECIO_HAIKU_OUTPUT, PRECIO_SONNET_INPUT, PRECIO_SONNET_OUTPUT, PRECIO_SONNET_CACHE_WRITE, PRECIO_SONNET_CACHE_READ) — se calcularon a partir del costo real registrado en las 30 llamadas de prueba y coinciden con la tarifa publicada por Anthropic para cada modelo.

Modelo	Input	Output	Cache write (5 min)	Cache read / hit
Claude Haiku 4.5	\$1 / MTok	\$5 / MTok	— (no se usa cache en clasificación)	—
Claude Sonnet 4.5	\$3 / MTok	\$15 / MTok	\$3.75 / MTok (1.25×)	\$0.30 / MTok (0.1×)

Claude Sonnet cuesta 3× más por token de entrada y 3× más por token de salida que Claude Haiku — de ahí que reservarlo solo para la tarea que realmente lo necesita (la redacción) sea la decisión de costos central de este proyecto.

3. Prompt caching: cómo funciona y cuánto ahorra en la práctica

El prompt de sistema de Claude Sonnet (instrucciones + base de conocimiento de FAQ) se marca con `cache_control: { type: "ephemeral" }`. La primera vez que ese bloque se envía dentro de una ventana de 5 minutos, Anthropic lo escribe al cache — eso cuesta más que el input normal (1.25×). Cualquier llamada posterior dentro de esa ventana de 5 minutos que reutilice el mismo bloque lo lee del cache a una fracción del precio (0.1×) en vez de volver a pagarlo como input completo.

Esto significa que el ahorro depende del tráfico: si los tickets se procesan con más de 5 minutos de diferencia entre sí, cada llamada a Sonnet paga la escritura de cache sin nadie que la reutilice — y esa llamada termina costando más, no menos, que si no se usara caching en absoluto. El ahorro aparece cuando hay ráfagas de tickets dentro de la misma ventana.

Caso	n	Costo real prom.	Costo sin caching (hipotético)	Ahorro prom.
Cache read (llamada reutiliza el cache)	12	\$0.00361	\$0.01101	+67.6%
Cache write (primera llamada de la ventana)	3	\$0.01393	\$0.01188	-17.3%
Total / promedio general (Sonnet)	15	\$0.00568	\$0.01118	+49.2%

En las pruebas de este proyecto, 12 de las 15 llamadas a Sonnet (80%) fueron cache hit — el caching resultó, en neto, en un ahorro del 49.2% sobre el costo de redacción.

4. Evidencia real: resultados de las pruebas de estrés

Estos números no son una estimación — son el promedio de las 30 llamadas reales a la API de Anthropic registradas en Airtable — Métricas IA durante las pruebas de estrés del workflow (15 tickets procesados de punta a punta, cada uno con una llamada a Haiku y una a Sonnet). El costo total de las 30 llamadas fue de \$0.0906 — menos de 9 centavos de dólar.

4.1 Claude Haiku 4.5 — Clasificación

Métrica	Valor
Llamadas registradas	15

Métrica	Valor
Tokens de entrada (promedio)	209
Tokens de salida (promedio)	31
Costo promedio por llamada	\$0.00036
Costo total (15 llamadas)	\$0.0054

4.2 Claude Sonnet 4.5 — Redacción

Métrica	Valor
Llamadas registradas	15
Tokens de entrada (promedio, ya neto de cache)	73
Tokens de salida (promedio)	183
Costo real promedio por llamada	\$0.00568
Costo total real (15 llamadas)	\$0.0852
Costo total si no se usara caching (hipotético)	\$0.1677
Ahorro total por prompt caching	\$0.0826 (49.2%)

Si la tarea de clasificación también se hiciera con Claude Sonnet en vez de Haiku, esas mismas 15 llamadas habrían costado \$0.0163 en lugar de los \$0.0054 reales — un 66.7% más caro por elegir el modelo equivocado para una tarea simple.

5. Proyección de costo a escala

Extrapolando el costo real promedio por ticket (\$0.00604 con caching, frente a \$0.01155 sin caching) a distintos volúmenes mensuales:

Volumen mensual de tickets	Costo real (con caching)	Costo sin caching	Ahorro estimado
100	\$0.60	\$1.15	\$0.55 (48%)
1.000	\$6.04	\$11.55	\$5.51 (48%)
10.000	\$60.41	\$115.46	\$55.05 (48%)

A 10.000 tickets/mes, usar Haiku (en vez de Sonnet) solo para la clasificación ahorra otros ~\$7.26/mes adicionales sobre esa proyección — la elección de modelo por tarea pesa tanto como el prompt caching.

6. Por qué no uso la Batch API (por ahora)

El triage de NubeShop necesita respuesta rápida por ticket: cada ticket dispara su propia clasificación y redacción, y la respuesta propuesta por la IA espera aprobación humana en Slack en cuestión de minutos.